

1 Manajemen Memori & System Call

Bayangkan meja kerja Anda. Di atas meja tersimpan dokumen yang sedang aktif digunakan — inilah yang diibaratkan sebagai **RAM**. Saat meja mulai penuh, beberapa dokumen dipindahkan ke laci agar ada ruang baru — inilah yang diibaratkan sebagai **swap**. Jika semua dokumen tersimpan di laci dan Anda terus bolak-balik mengambilnya, pekerjaan menjadi lambat. Itulah gambaran sederhana bagaimana sistem operasi Linux mengelola memori.

Pada bab ini kita akan memahami konsep tersebut lebih dalam, sekaligus mempelajari **system call** — mekanisme yang digunakan program untuk meminta layanan dari kernel. Kita juga akan memanfaatkan kemampuan Bash dari Bab 9 untuk membangun script monitoring yang nyata dan berguna.

Tujuan Praktikum

Setelah menyelesaikan praktikum ini, mahasiswa diharapkan mampu:

1. memahami arsitektur memori pada Linux;
2. menjelaskan konsep memori virtual dan mekanisme paging;
3. mengelola dan mengonfigurasi swap space;
4. memantau penggunaan memori menggunakan berbagai alat;
5. memahami konsep dan jenis-jenis system call;
6. mengamati interaksi antara user space dan kernel space.

Catatan

Seluruh praktikum dijalankan sebagai pengguna biasa kecuali perintah yang secara eksplisit menggunakan **sudo**. Buat direktori kerja terlebih dahulu:

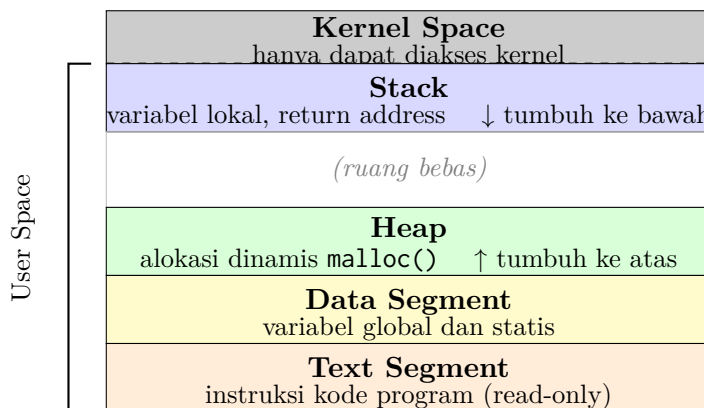
```
mkdir -p ~/praktikum-os/week10-memory
```

1.1 Arsitektur Memori di Linux

Penjelasan Konsep

Setiap program yang berjalan di Linux mendapatkan *ruang alamat virtual* sendiri. Artinya, setiap program seolah-olah memiliki memori pribadi yang tidak dapat diganggu program lain. Kernel bertugas mengatur pemetaan antara alamat virtual tersebut dengan memori fisik (RAM).

Ruang alamat virtual sebuah proses dibagi menjadi beberapa area dengan fungsi berbeda, seperti terlihat pada gambar 1.1.



Gambar 1.1: Peta memori virtual sebuah proses Linux. Setiap proses memiliki ruang alamat virtual sendiri yang diisolasi dari proses lain.

Catatan

Isolasi antar proses menjamin stabilitas sistem. Jika satu program crash, kerusakan hanya berdampak pada proses tersebut — proses lain tetap berjalan normal karena tidak dapat saling mengakses memori satu sama lain.

Perintah yang Digunakan

- `free -h`: ringkasan penggunaan RAM dan swap dalam format mudah dibaca.
- `cat /proc/meminfo`: detail informasi memori langsung dari kernel.
- `top`: memantau penggunaan resource sistem secara real-time.

Praktikum 10.1 Melihat Penggunaan Memori

Tujuan: mengenali struktur dan penggunaan memori pada sistem Linux.

Langkah 1: Jalankan `free -h` untuk melihat ringkasan RAM dan swap.

```
free -h
```

Kode 1.1: Melihat penggunaan memori

Output `free -h` menampilkan dua baris: **Mem** untuk RAM dan **Swap** untuk swap. Kolom penting pada baris **Mem**:

- **total**: jumlah total RAM yang terpasang.
- **used**: memori yang sedang digunakan proses dan kernel.
- **free**: memori yang benar-benar kosong, belum dipakai sama sekali.
- **available**: estimasi memori yang *benar-benar tersedia* untuk aplikasi baru, termasuk cache yang bisa dibebaskan. Gunakan nilai ini, bukan **free**, untuk menilai kondisi memori sistem.
- **buff/cache**: memori yang digunakan kernel sebagai buffer I/O dan cache file. Linux membebaskan cache ini secara otomatis jika ada aplikasi yang membutuhkan.

Langkah 2: Lihat detail memori dari kernel melalui `/proc/meminfo`.

```
cat /proc/meminfo | head -n 20
```

Kode 1.2: Melihat detail memori kernel

Analisis:

1. Hitung persentase memori tersedia: $\text{available} / \text{total} \times 100\%$. Jika hasilnya di bawah 10%, sistem mulai kekurangan memori.
2. Pada baris **Swap**, apakah kolom **used** bernilai 0? Jika lebih dari 0, kernel sudah pernah memindahkan data ke disk karena RAM tidak cukup.
3. Perhatikan field **Cached** dan **Buffers** di `/proc/meminfo`. Nilai ini sesuai dengan kolom **buff/cache** pada `free -h`.

Tips

Jangan khawatir jika nilai **used** tampak besar. Linux sengaja menggunakan RAM yang kosong sebagai cache untuk mempercepat akses file. Fokus pada **available** — inilah memori yang sesungguhnya tersedia.

Studi Kasus 10.1 Server Lambat karena Memori

Skenario: Server aplikasi terasa lambat saat banyak pengguna aktif. Administrator perlu menentukan apakah penyebabnya adalah kekurangan memori.

Langkah 1: Periksa kondisi memori secara keseluruhan.

```
free -h
```

Kode 1.3: Pemeriksaan awal kondisi memori

Langkah 2: Pantau proses secara real-time.

```
top
```

Kode 1.4: Pemantauan real-time dengan top

Di dalam **top**: tekan **M** untuk mengurutkan berdasarkan memori, tekan **q** untuk keluar.

Analisis:

1. Apakah nilai **available** sangat kecil (misalnya di bawah 200 MB pada server dengan RAM 2 GB)? Jika ya, server kemungkinan kekurangan memori.
2. Apakah kolom **used** pada baris **Swap** lebih dari 0? Jika ya, kernel sedang menggunakan swap, yang berarti performa menurun.
3. Di tampilan **top**, proses apa yang memiliki **%MEM** terbesar? Proses tersebut menjadi kandidat utama penyebab lambatnya server.

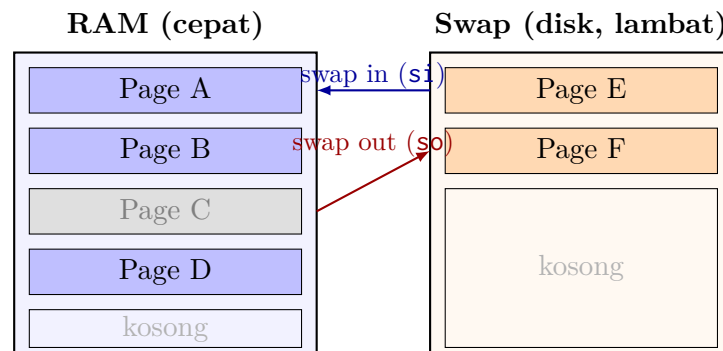
1.2 Manajemen Memori Virtual

Penjelasan Konsep

Bayangkan sebuah perpustakaan besar dengan ribuan buku. Anda hanya dapat meletakkan beberapa buku di meja baca (RAM), sementara sisanya tersimpan di rak (disk/swap). Saat Anda butuh buku dari rak, Anda harus mengambilnya dan mungkin menyimpan buku lain kembali ke rak. Itulah cara kerja *memori virtual* dan *paging*.

Memori virtual memungkinkan program menggunakan lebih banyak memori daripada RAM yang tersedia. Data dibagi dalam blok berukuran tetap yang disebut **page**. Kernel secara otomatis memindahkan page yang tidak aktif ke swap (*swap*

out) dan memuatnya kembali saat dibutuhkan (*swap in*), seperti digambarkan pada gambar 1.2.



Gambar 1.2: Mekanisme paging: kernel memindahkan page tidak aktif ke swap saat RAM penuh, dan memuatnya kembali (*si/so* pada *vmstat*) saat dibutuhkan.

Perintah yang Digunakan

- `vmstat 1 5`: statistik memori virtual, CPU, dan I/O setiap 1 detik sebanyak 5 kali. Kolom *si* (swap in) dan *so* (swap out) adalah indikator utama aktivitas paging.

Praktikum 10.2 Mengamati Aktivitas Paging

Tujuan: memahami aktivitas memori virtual melalui kolom swap pada *vmstat*.

Langkah 1: Jalankan *vmstat* dengan interval 1 detik, 5 sampel.

```
vmstat 1 5
```

Kode 1.5: Melihat statistik memori virtual

Pada output, temukan kolom **swap** yang berisi dua sub-kolom:

- **si**: KB/detik yang dipindahkan dari disk ke RAM. Nilai 0 = tidak ada data yang sedang dimuat dari swap.
- **so**: KB/detik yang dipindahkan dari RAM ke disk. Nilai 0 = RAM masih cukup, tidak ada data yang dikeluarkan ke swap.

Analisis:

1. Amati nilai **si** dan **so** pada kelima baris. Pada sistem normal dengan RAM cukup, kedua nilai ini selalu 0.

1.3 Konfigurasi Swap Space

2. Jika nilai **si** atau **so** sesekali muncul lebih dari 0, artinya pernah ada aktivitas swap. Ini masih wajar jika tidak terus-menerus.
3. Jika **si** dan **so** terus-menerus lebih dari 0, sistem dalam kondisi *memory pressure* serius — performa turun drastis karena akses disk jauh lebih lambat dari RAM.
4. Perhatikan juga kolom **free** (RAM kosong) dan **buff** (buffer) untuk memahami kondisi keseluruhan RAM saat itu.

Peringatan

Nilai **si** dan **so** yang tinggi terus-menerus menandakan sistem butuh lebih banyak RAM. Solusi: tambah RAM, kurangi jumlah proses yang berjalan bersamaan, atau optimalkan konfigurasi aplikasi.

1.3 Konfigurasi Swap Space

Penjelasan Konsep

Swap space adalah area di disk yang berfungsi sebagai perpanjangan RAM. Swap mencegah sistem crash saat RAM penuh, tetapi karena berada di disk, aksesnya jauh lebih lambat. Swap adalah *jaring pengaman*, bukan pengganti RAM.

Parameter **swappiness** (nilai 0–100) mengatur agresivitas kernel dalam menggunakan swap:

- **Nilai tinggi (60–100)**: kernel cepat memindahkan data ke swap meski RAM masih tersedia. Ini nilai default Linux (60).
- **Nilai rendah (10–20)**: kernel lebih mengutamakan RAM dan hanya menggunakan swap jika benar-benar terpaksa. Disarankan untuk server aplikasi.

Perintah yang Digunakan

- `swapon -show`: daftar swap aktif beserta ukuran dan penggunaannya.
- `fallocate -l ukuran file`: membuat file kosong berukuran tertentu secara efisien.
- `mkswap file`: memformat file sebagai area swap.
- `swapon/swapoff`: mengaktifkan/menonaktifkan swap.
- `sysctl vm.swappiness=nilai`: mengubah swappiness sementara.

Praktikum 10.3 Membuat dan Mengonfigurasi Swap File

Tujuan: menambahkan swap space dan memahami parameter swappiness.

Langkah 1: Buat file berukuran 512 MB sebagai calon swap.

```
sudo fallocate -l 512M /swapfile-week10
```

Kode 1.6: Membuat swap file 512MB

Langkah 2: Atur permission file menjadi **600** — hanya root yang boleh membaca dan menulis.

```
sudo chmod 600 /swapfile-week10
```

Kode 1.7: Mengamankan permission swap file

Peringatan

Langkah ini **wajib** dilakukan sebelum **mkswap**. File swap dapat berisi potongan data sensitif dari memori program. Permission terbuka (644) membuat pengguna lain bisa membaca isi memori tersebut.

Langkah 3: Format file sebagai area swap, lalu aktifkan.

```
sudo mkswap /swapfile-week10
sudo swapon /swapfile-week10
```

Kode 1.8: Memformat dan mengaktifkan swap

Langkah 4: Verifikasi swap aktif. Anda akan melihat entri **/swapfile-week10** dengan ukuran 512M, dan nilai **total** pada baris **Swap** di **free -h** bertambah 512M.

```
swapon --show
free -h
```

Kode 1.9: Verifikasi swap aktif

Langkah 5: Periksa nilai swappiness, ubah sementara, dan verifikasi perubahan.

```
cat /proc/sys/vm/swappiness
sudo sysctl vm.swappiness=10
cat /proc/sys/vm/swappiness
```

Kode 1.10: Melihat dan mengubah swappiness

Analisis:

1. Berapa nilai swappiness default? Apa artinya bagi perilaku kernel dalam menggunakan swap?
2. Setelah diubah ke 10, konfirmasi nilai berubah pada output `cat` kedua. Apa dampak nilai 10 terhadap penggunaan swap dibanding nilai 60?
3. Apakah entri `/swapfile-week10` muncul di `swapon -show`? Jika tidak, pastikan Langkah 2 (`chmod 600`) sudah dijalankan sebelum Langkah 3.

Catatan

Perubahan `sysctl` bersifat sementara dan hilang setelah reboot. Untuk konfigurasi permanen, tambahkan `vm.swappiness=10` pada `/etc/sysctl.conf`. Bersihkan swap file setelah praktikum:

```
sudo swapoff /swapfile-week10
sudo rm /swapfile-week10
```

1.4 Pemantauan Penggunaan Memori

Penjelasan Konsep

Pemantauan memori membantu administrator mendeteksi masalah sebelum berdampak ke pengguna. Ada dua pendekatan:

- **Real-time** dengan `top`: melihat perubahan dinamis yang terjadi saat ini.
- **Snapshot** dengan `ps aux`: mengambil daftar proses pada satu titik waktu, cocok disimpan ke file laporan.

Kolom penting pada output `ps aux`:

- **%MEM**: persentase RAM yang digunakan proses terhadap total RAM.
- **RSS** (Resident Set Size): ukuran RAM fisik yang *benar-benar dipakai* proses saat ini (KB).
- **VSZ** (Virtual Memory Size): total memori virtual yang dialokasikan, termasuk bagian yang belum dimuat ke RAM (selalu lebih besar dari RSS).

Praktikum 10.4 Monitoring Memory

Tujuan: mengidentifikasi proses dengan penggunaan memori terbesar.

Langkah 1: Ambil snapshot proses diurutkan dari penggunaan memori terbesar.

```
ps aux --sort=-%mem | head
```

Kode 1.11: Proses dengan memori terbesar

Baris pertama adalah header (nama kolom). Proses di baris kedua adalah pengguna memori terbesar saat perintah dijalankan.

Langkah 2: Pantau secara real-time dengan **top**.

```
top
```

Kode 1.12: Monitoring real-time

Di dalam **top**: **[M]** = urutkan berdasarkan memori, **[P]** = urutkan berdasarkan CPU, **[q]** = keluar.

Analisis:

1. Proses apa yang berada di urutan pertama? Catat nilai %MEM dan RSS-nya.
2. Konversikan RSS dari KB ke MB (bagi 1024). Misalnya, RSS=524288 berarti proses menggunakan 512 MB RAM. Apakah wajar untuk jenis program tersebut?
3. Mengapa VSZ selalu lebih besar dari RSS pada proses yang sama?
4. Apakah urutan proses di **ps** konsisten dengan tampilan **top** saat diurutkan berdasarkan %MEM?

Tips

Kombinasikan tiga alat: **free** untuk kondisi global, **top** untuk dinamika real-time, dan **ps** untuk snapshot yang dapat disimpan ke file.

Praktikum 10.5 Script Monitor Memori

Tujuan: mengotomasi pemantauan memori menggunakan Bash script.

Catatan

Script ini menerapkan konsep dari Bab 9: variabel (**THRESHOLD**), kondisi **if**, parsing output dengan **awk**, dan perintah **date** untuk mencatat waktu pemantauan.

Masuk ke direktori kerja dan buat file script:

```
cd ~/praktikum-os/week10-memory  
nano monitor-memori.sh
```

Kode 1.13: Pindah ke direktori dan buka nano

Ketik script berikut:

```
#!/bin/bash  
set -euo pipefail  
  
THRESHOLD=20  
  
echo "=== Monitor Memori ==="  
date  
echo  
  
free -h  
echo  
  
AVAIL=$(free | awk '/Mem/ {printf "%d", $7/$2*100}')  
if [ "$AVAIL" -lt "$THRESHOLD" ]; then  
    echo "PERINGATAN: Memori tersedia hanya ${AVAIL}%!"  
else  
    echo "Status: Memori tersedia ${AVAIL}% (normal)"  
fi  
echo  
  
echo "--- 5 Proses Memori Tertinggi ---"  
ps aux --sort=-%mem | head -n 6 | tail -n 5
```

Kode 1.14: Isi file monitor-memori.sh

Catatan

Simpan: **Ctrl+O** → **Enter** → keluar: **Ctrl+X**.

```
chmod +x monitor-memori.sh  
bash monitor-memori.sh
```

Kode 1.15: Menjalankan script monitor

Analisis:

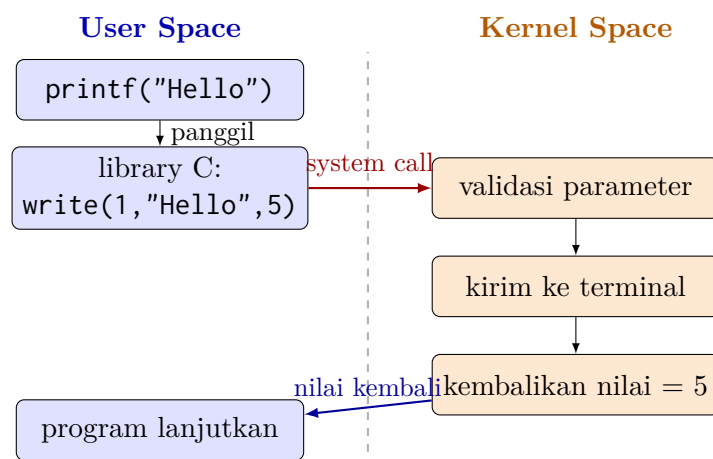
1. Variabel `THRESHOLD=20` menetapkan batas persentase. Perintah `free | awk '/Mem/ {printf "%d", $7/$2*100}'` mengambil kolom ke-7 (available) dibagi kolom ke-2 (total) dari baris `Mem`, lalu dikalikan 100 untuk menghasilkan persentase bilangan bulat.
2. Kondisi `if [ "$AVAIL" -lt "$THRESHOLD" ]` bernilai benar jika persentase memori tersedia di bawah 20.
3. Ubah `THRESHOLD` menjadi `90` dan jalankan ulang. Apa yang berubah pada output? Mengapa demikian?

1.5 System Call dan Interaksi User-Kernel

Penjelasan Konsep

Bayangkan sebuah restoran. Anda (program) tidak bisa langsung masuk ke dapur (kernel) untuk memasak sendiri — Anda harus memesan melalui pramusaji (system call). Pramusaji meneruskan pesanan ke dapur, memastikan pesanan valid, dan membawa hasilnya kembali ke meja Anda.

Itulah cara kerja system call: mekanisme bagi program di *user space* untuk meminta layanan dari kernel. Program tidak dapat langsung mengakses hardware, filesystem, atau sumber daya kernel — semua permintaan harus melalui system call. Alur lengkapnya ditunjukkan pada gambar 1.3.



Gambar 1.3: Alur system call `printf`: program user meminta layanan kernel melalui antarmuka yang terkontrol, bukan langsung ke hardware.

1.5.1 Jenis-Jenis System Call

System call dikelompokkan berdasarkan fungsinya:

- **Manajemen Proses:** `fork`, `exec`, `wait`, `exit` — membuat, menjalankan, menunggu, dan mengakhiri proses.
- **Manajemen File:** `open`, `read`, `write`, `close` — membuka, membaca, menulis, dan menutup file.
- **Manajemen Memori:** `mmap`, `brk` — memetakan file ke memori dan mengatur batas heap.
- **Komunikasi:** `pipe`, `socket`, `send`, `recv` — komunikasi antar proses dan jaringan.
- **Informasi Sistem:** `getpid`, `uname`, `time` — mengambil informasi proses dan sistem.

Catatan

Perintah sederhana seperti `ls` sebenarnya memanggil puluhan system call: `openat` (buka direktori), `getdents64` (baca isi), `newfstatat` (baca metadata), `write` (tampilkan ke terminal), dan masih banyak lagi.

Studi Kasus 10.2 Gagal Akses File

Skenario: Program tidak dapat membaca file konfigurasi. Penyebab umum: file tidak ada, path salah, atau permission tidak sesuai. Kita akan mensimulasikan kondisi ini dan mengamati pesan error yang dihasilkan.

Langkah 1: Buat direktori dan file konfigurasi contoh.

```
mkdir -p ~/praktikum-os/week10-memory/syscall-case
cd ~/praktikum-os/week10-memory/syscall-case
echo "PORT=8080" > app.conf
ls -l app.conf
cat app.conf
```

Kode 1.16: Membuat file konfigurasi contoh

Langkah 2: Simulasikan permission bermasalah.

```
chmod 000 app.conf
cat app.conf
```

Kode 1.17: Menghapus semua izin akses

Output akan menampilkan: `cat: app.conf: Permission denied`. Ini terjadi karena system call `openat()` gagal — kernel menolak permintaan membuka file karena tidak ada bit izin baca (`r`).

Langkah 3: Kembalikan permission dan verifikasi.

```
chmod 644 app.conf
cat app.conf
```

Kode 1.18: Mengembalikan permission file

Analisis:

1. Mengapa `cat` menghasilkan `Permission denied` setelah `chmod 000`? System call apa yang gagal?
2. Apa perbedaan pesan error `Permission denied` vs `No such file or directory`? Coba `rm app.conf` lalu `cat app.conf` untuk melihat perbedaannya.
3. Permission `644` berarti apa untuk owner, group, dan others?

Peringatan

Pesan error seperti `Permission denied` berasal dari system call yang gagal di level kernel. Sebelum debugging lebih jauh, selalu verifikasi dengan `ls -l` atau `stat namafile`.

Praktikum 10.6 Mengamati System Call dengan strace

Tujuan: melihat dan menganalisis system call yang dilakukan suatu perintah.

Perintah yang Digunakan:

- `strace` perintah: jalankan perintah sambil catat semua system call (argumen dan nilai kembalian).
- `strace -c` perintah: tampilkan ringkasan statistik (jumlah pemanggilan, waktu, error).

Langkah 1: Lihat 30 baris pertama system call dari perintah `ls`.

```
strace ls 2>&1 | head -n 30
```

Kode 1.19: Detail system call ls

Setiap baris mengikuti format: `namasyscall(argumen) = nilai_kembali`. Contoh interpretasi:

1.6 Tugas Praktikum

- `openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY) = 3`: membuka file untuk dibaca, berhasil (file descriptor = 3).
- `read(3, "...", 4096) = 1234`: membaca hingga 4096 byte dari fd 3, berhasil membaca 1234 byte.
- `close(3) = 0`: menutup fd 3, berhasil.
- `openat(. . .) = -1 ENOENT`: gagal karena file tidak ditemukan.

Langkah 2: Lihat ringkasan statistik dan bandingkan dua direktori berbeda.

```
strace -c ls
strace -c ls /etc 2>&1 | tail -5
```

Kode 1.20: Ringkasan dan perbandingan system call

Analisis:

1. Dari output Langkah 1, identifikasi minimal 4 system call berbeda. Jelaskan fungsi singkat masing-masing berdasarkan argumen yang terlihat.
2. Dari ringkasan `strace -c`, system call mana yang paling sering dipanggil? Mengapa?
3. Apakah ada system call dengan `errors` lebih dari 0? Apakah itu berarti program bermasalah, ataukah bagian normal dari logika program?
4. Apakah jumlah system call berbeda antara `ls` dan `ls /etc`? Faktor apa yang menyebabkan perbedaan tersebut?

Tips

Gunakan `strace -c` untuk gambaran umum cepat. Untuk mendiagnosis masalah konkret (misalnya program tidak bisa buka file tertentu), gunakan `strace` tanpa `-c` agar dapat melihat urutan dan argumen lengkap setiap system call.

1.6 Tugas Praktikum

Instruksi Umum: Kerjakan seluruh tugas pada direktori berikut.

```
mkdir -p ~/praktikum-os/week10-memory
cd ~/praktikum-os/week10-memory
```

Kode 1.21: Persiapan direktori tugas

Tugas 10.1 Audit Penggunaan Memori Sistem

Instruksi: Buat script `memory-audit.sh` yang menghasilkan laporan kondisi memori sistem secara otomatis.

```
nano ~/praktikum-os/week10-memory/memory-audit.sh
```

Kode 1.22: Buka file dengan nano

```
#!/bin/bash
set -euo pipefail

LAPORAN="memory-report.txt"

{
    echo "=== LAPORAN MEMORI SISTEM ==="
    date
    echo
    echo "--- Ringkasan free -h ---"
    free -h
    echo
    echo "--- /proc/meminfo ---"
    cat /proc/meminfo | head -n 20
} > "$LAPORAN"

echo "Laporan disimpan ke: $LAPORAN"
cat "$LAPORAN"
```

Kode 1.23: Isi file memory-audit.sh

Catatan

Simpan: `Ctrl+O` → `Enter` → keluar: `Ctrl+X`.

```
chmod +x ~/praktikum-os/week10-memory/memory-audit.sh
cd ~/praktikum-os/week10-memory
bash memory-audit.sh
```

Kode 1.24: Jalankan script audit

Analisis

1. Hitung persentase memori tersedia (`available / total × 100%`). Apakah sistem dalam kondisi normal?

1.6 Tugas Praktikum

2. Mengapa **buff/cache** tidak dihitung sebagai memori yang *terpakai* dari sudut pandang ketersediaan untuk aplikasi?
3. Dari `/proc/meminfo`, apakah **SwapTotal** lebih besar dari 0? Berapa nilai **SwapFree**?

Tugas 10.2 Identifikasi Proses dengan Memori Tertinggi

Instruksi: Simpan daftar 10 proses pengguna memori terbesar ke file.

```
ps aux --sort=-%mem | head -n 10 > top-memory-process.txt
cat top-memory-process.txt
```

Kode 1.25: Simpan dan tampilkan proses teratas

Analisis

1. Proses apa di urutan pertama? Catat nilai **%MEM** dan **RSS**.
2. Konversikan **RSS** ke **MB** (bagi 1024). Apakah wajar?
3. Jumlahkan **%MEM** dari 5 proses teratas. Berapa persen **RAM** yang mereka gunakan bersama?

Tugas 10.3 Membuat dan Memverifikasi Swap File

Instruksi: Buat swap file khusus tugas sebesar 256 MB dan verifikasi.

```
sudo fallocate -l 256M /swapfile-tugas-week10
sudo chmod 600 /swapfile-tugas-week10
sudo mkswap /swapfile-tugas-week10
sudo swapon /swapfile-tugas-week10
```

Kode 1.26: Buat dan aktifkan swap file tugas

```
{
    echo "=== VERIFIKASI SWAP ==="
    swapon --show
    echo
    free -h
} > swap-check.txt
cat swap-check.txt
```

Kode 1.27: Verifikasi dan simpan hasil

Catatan

Bersihkan setelah selesai:

```
sudo swapoff /swapfile-tugas-week10 && sudo rm
/swapfile-tugas-week10
```

Analisis

1. Identifikasi kolom NAME, TYPE, SIZE, dan USED pada output `swapon -show`.
2. Apakah nilai **total** pada baris Swap di `free -h` bertambah 256 MB?
3. Mengapa permission **600** penting? Apa risiko jika diatur ke 644?

Tugas 10.4 Analisis System Call dengan strace

Instruksi: Analisis system call yang dipanggil perintah `ls`.

```
strace -c ls 2> strace-summary.txt
strace ls /etc 2> strace-ls-etc.txt
cat strace-summary.txt
```

Kode 1.28: Simpan ringkasan dan detail system call

Analisis

1. Sebutkan minimal 5 system call dari `strace-summary.txt` beserta fungsi singkatnya.
2. System call mana yang paling sering dipanggil? Mengapa?
3. Apakah ada **errors** lebih dari 0? Apakah program tetap berjalan normal meskipun ada kegagalan tersebut?

Tugas 10.5 Studi Kasus Diagnosa Server Lambat

Skenario: Server terasa lambat. Buat script diagnosa yang menggabungkan semua pemeriksaan dari bab ini menggunakan fungsi Bash.

```
nano ~/praktikum-os/week10-memory/diagnosa-server.sh
```

Kode 1.29: Buka file dengan nano

```
#!/bin/bash
set -euo pipefail
```

```
LAPORAN="diagnosa-server-lambat.txt"
WARN_MEM=false
WARN_SWAP=0

cek_memori() {
    echo "--- Kondisi Memori ---"
    free -h
    echo
    AVAIL_PCT=$(free | awk '/Mem/ {printf "%d", $7/$2*100}'
    ')
    if [ "$AVAIL_PCT" -lt 20 ]; then
        echo "PERINGATAN: Memori tersedia hanya ${
    AVAIL_PCT}%"
        WARN_MEM=true
    fi
}

cek_swap() {
    echo "--- Penggunaan Swap ---"
    swapon --show 2>/dev/null || echo "Tidak ada swap
aktif"
    echo
    WARN_SWAP=$(free | awk '/Swap/ {print $3}')
    if [ "$WARN_SWAP" -gt 0 ]; then
        echo "INFO: Swap digunakan (${WARN_SWAP} kB)"
    fi
}

cek_proses() {
    echo "--- 10 Proses Memori Tertinggi ---"
    ps aux --sort=-%mem | head -n 11
    echo
}

cek_paging() {
    echo "--- Aktivitas Paging (5 sampel) ---"
    vmstat 1 5
    echo
}

ringkasan() {
```

```

echo "=== RINGKASAN ==="
if [ "$WARN_MEM" = true ]; then
    echo "- Memori: KRITIS - perlu tindakan segera"
else
    echo "- Memori: normal"
fi
if [ "$WARN_SWAP" -gt 0 ]; then
    echo "- Swap: aktif - pantau aktivitas paging"
else
    echo "- Swap: tidak digunakan"
fi
}

{
    echo "=== LAPORAN DIAGNOSA SERVER ==="
    date
    echo

    cek_memori
    cek_swap
    cek_proses
    cek_paging
    ringkasan
} | tee "$LAPORAN"

echo
echo "Laporan disimpan ke: $LAPORAN"

```

Kode 1.30: Isi file diagnosa-server.sh

Catatan

Simpan: **Ctrl+O** → **Enter** → keluar: **Ctrl+X**.

```

chmod +x ~/praktikum-os/week10-memory/diagnosa-server.sh
cd ~/praktikum-os/week10-memory
bash diagnosa-server.sh

```

Kode 1.31: Jalankan script diagnosa

Analisis

1.7 Referensi

1. Jelaskan peran masing-masing fungsi: `cek_memori`, `cek_swap`, `cek_proses`, `cek_paging`, dan `ringkasan`. Mengapa diagnosa dipecah menjadi fungsi terpisah?
2. Berdasarkan bagian `RINGKASAN`, apakah kondisi sistem normal atau kritis? Jelaskan berdasarkan nilai threshold yang digunakan script.
3. Mengapa script menggunakan `tee "$LAPORAN"` bukan redirection biasa `> "$LAPORAN"`? Apa keuntungannya?
4. Dari output `cek_paging`, apakah ada aktivitas `si` atau `so`? Jika ada, apa implikasinya terhadap performa server?

1.7 Referensi

- Nemeth et al., *UNIX and Linux System Administration Handbook*.
- Dokumentasi Linux Kernel (`/proc`, `man 2 syscall`).
- Manual page: `man free`, `man vmstat`, `man swapon`, `man strace`, `man 2 syscall`.